Sutra Benchmark — Consolidated

Same A/B (index vs grep) across three repos spanning 40× in size. What the first run implied, and what two real repos actually showed.

2026-07-08 · sutra 1.9k · frappe 13.4k · dify 75k symbols · claude-sonnet · 5 tickets each · checkouts aligned to indexed commits

−18 → −49%

fewer **agentic turns** — grows with repo size

the cleanest, monotonic signal

+10 → −46%

wall-clock — penalty on tiny repo → big win on large

flips and widens with size

+28 / +56%

more **fresh tokens** on real repos

the −54% token "win" did NOT generalize

= ×3

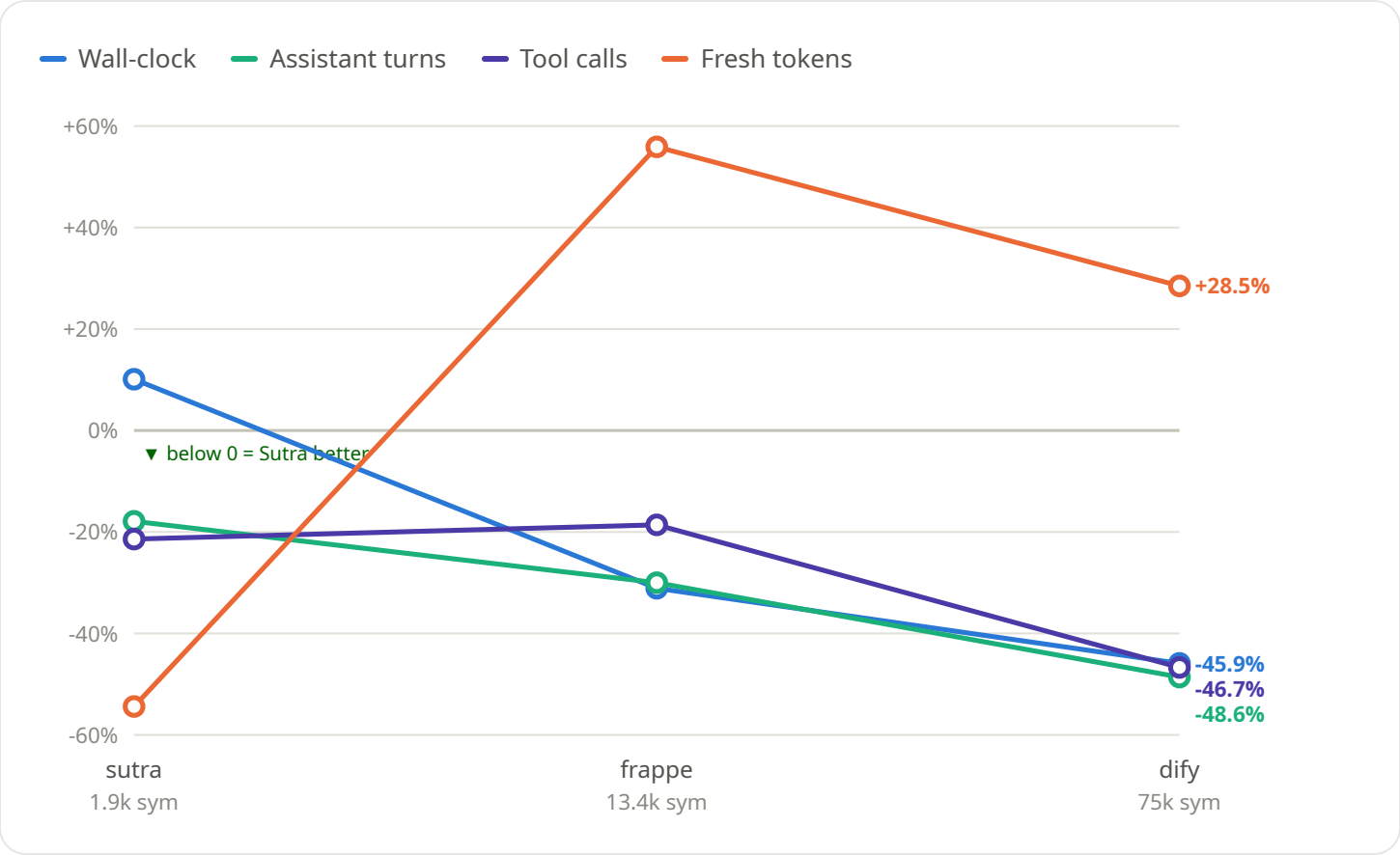
equal quality on every run

no correctness traded for speed

Corrected conclusion: the first run (tiny self-repo) suggested Sutra saves ~54% of tokens. Two larger real repos did not reproduce that — Sutra used **more** tokens on both. The honest takeaway: **Sutra is a speed-and-steps optimization, not a token optimization.** It reliably cuts wall-clock and agentic turns on real codebases (and the win grows with size), but its dense semantic-search payloads cost more tokens than terse grep output.

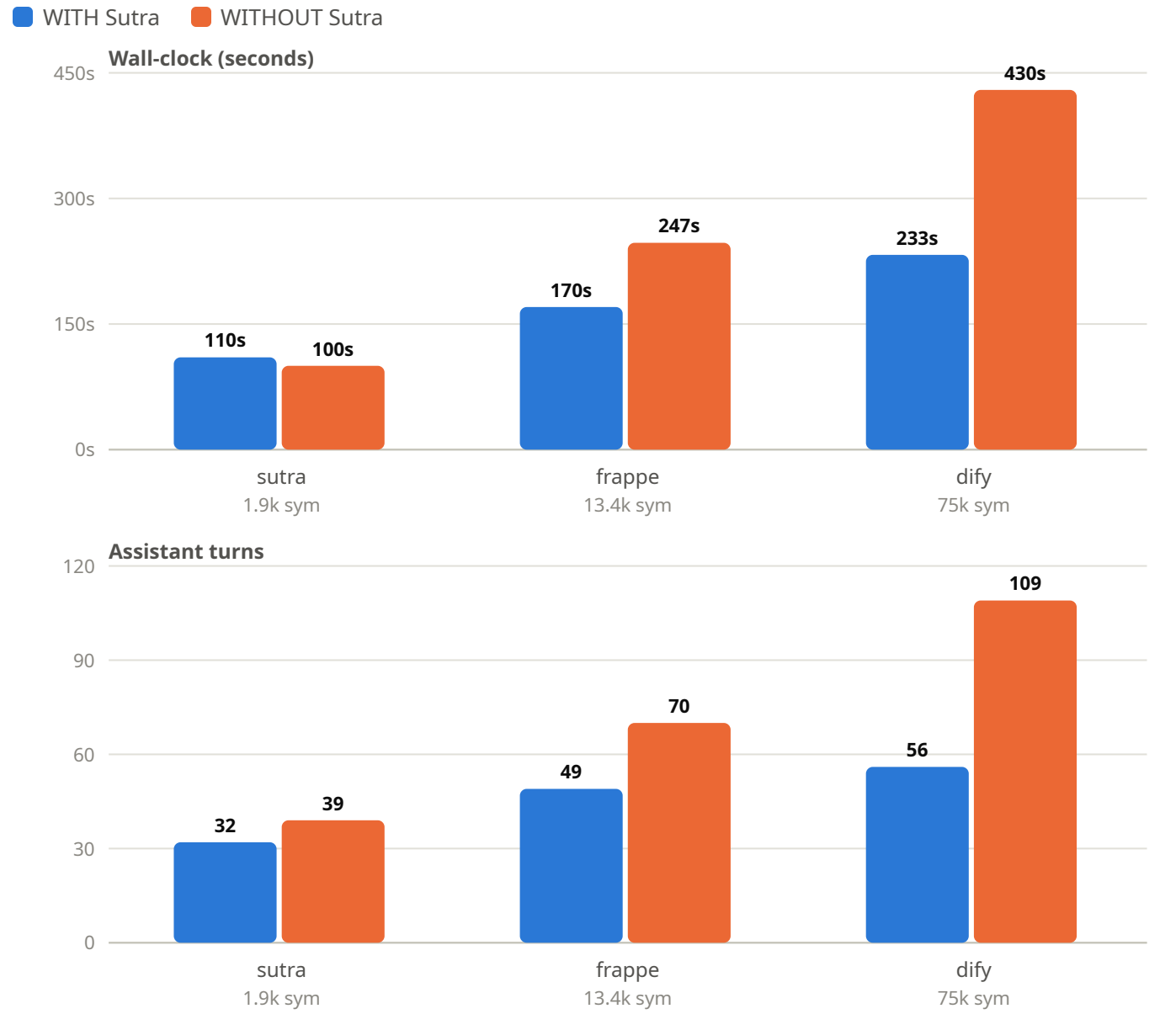
Scaling: Sutra's effect as the repo grows

Each line = one metric's % change vs grep-only, across the three repo sizes. **Below the zero line = Sutra better.** Turns and wall-clock trend steadily down (wins grow); fresh tokens jump from a −54% win on the tiny repo to a cost on both real ones.



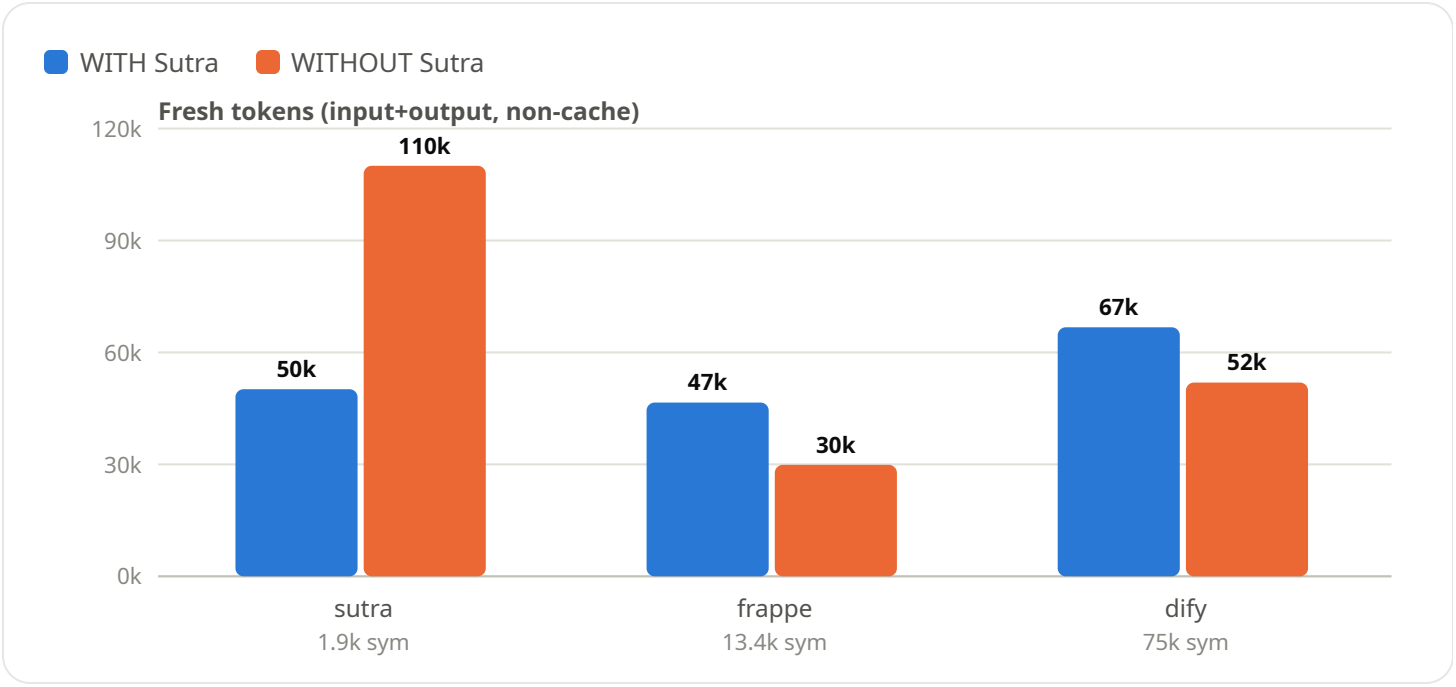
The clean wins — wall-clock & turns (per repo)

The two metrics that hold up monotonically. On both real repos Sutra roughly cut wall-clock by a third to a half, and did it in far fewer turns.



The honest cost — fresh tokens (per repo)

Only the tiny self-repo (left) favored Sutra on tokens, because the grep agent read whole files there. On both real repos Sutra's search payloads cost more.



All measured figures

Metric	sutra A	sutra B	eff	frappe A	frappe B	eff	dify A	dify B	eff
Reported tokens	73,480	79,538	**-8%**	89,533	71,629	**+25%**	126,881	115,465	**+10%**
Fresh tokens	50,145	110,026	**-54%**	46,569	29,872	**+56%**	66,739	51,929	**+29%**
Total context	1,668,789	2,201,018	**-24%**	2,942,652	3,387,222	**-13%**	4,507,149	7,978,895	**-44%**
Tool calls	22	28	**-21%**	35	43	**-19%**	40	75	**-47%**
Assistant turns	32	39	**-18%**	49	70	**-30%**	56	109	**-49%**
Files read	11	15	**-27%**	20	23	**-13%**	13	19	**-32%**
Wall-clock (s)	110.2	100.1	**+10%**	170.3	247.0	**-31%**	232.6	429.6	**-46%**

A = WITH Sutra, B = WITHOUT. "eff" = Sutra effect (green = used less, red = used more).

Caveats

- **n = 1 per repo** — directional, not statistical. The fact that only *turns* and *wall-clock* are strictly monotonic (tool calls, total-context and tokens bounce in magnitude) is itself the sign that per-run variance is real. A defensible claim needs ≥3 trials per repo with medians.
- **Earlier "monotonic across the board" was overstated** — that read came from only two points (sutra→dify). The frappe midpoint shows tool-calls and total-context are directionally consistent but not clean lines.
- **Equal-quality is qualitative** — both agents were independently correct and deep; on two runs each side found a real bug the other missed. No separate grader scored them.
- **Token lenses differ** — `subagent_tokens` weights cache opaquely; fresh (non-cache input+output) best reflects model work; total-context is cache-dominated and least comparable.
- **frappe source** was a fresh shallow clone in the scratchpad aligned to the indexed commit; the off-limits `first-bench` copy was never touched (verified from transcripts).